A.S.H.R.A

Product Requirements Document

A private, permissioned personal assistant that reads your email and calendar, keeps your tasks and commitments, drafts in your voice, and reaches you on Telegram — while sensitive data never leaves your machine.

WHAT THIS DOCUMENT DEFINES

01 · Product

Vision, scope, user journeys, and the hard boundary of v1.0.

02 · Trust

Privacy classes, model lanes, approvals, credentials, and retention.

03 · System

Connectors, life model, tasks, subagents, interfaces, and the roadmap.

VERSION	DATE	STATUS	OWNER
1.0 — Draft	13 November 2026	For review	Lohit

LOCAL-ONLY DOCUMENT · NOT PUBLISHED · WINDOWS 11 · SINGLE USER

Document control

DOCUMENT	Product Requirements Document	PRODUCT	A.S.H.R.A
VERSION	1.0 — Draft	DATE	13 November 2026
STATUS	For review and approval	CLASSIFICATION	Local / private
PLATFORM	Windows 11, single user	BASELINE	v0.7.1 · commit 5b774ac

Purpose of this document

This PRD defines what A.S.H.R.A is, what the first usable release must do, how it is allowed to behave with private data, and how the work is sequenced. It is written to be sufficient on its own: an engineer should be able to build from it, and the owner should be able to accept or reject the result against it.

It reflects decisions made during product discovery on 12–13 November 2026, including the choice of Gmail IMAP, Google Calendar read-only via private iCal, Telegram as the first control channel, a local task ledger, and a local model for sensitive content.

How to read the requirements

MARKER	MEANING
MUST	Required for v1.0. A release without it is not shippable.
SHOULD	Expected in v1.0; may slip one milestone with an explicit note.
COULD	Valuable, deferred unless it is cheap and low-risk.
LOCAL	Runs on this machine. Content must not leave it.
CLOUD	May use the configured cloud model; never carries private content.
NEVER	Out of scope by policy, not by scheduling.

Revision history

VERSION	DATE	CHANGE
0.1	12 Nov 2026	Discovery notes: assistant scope, connectors, autonomy, privacy.
0.5	13 Nov 2026	Locked email, calendar, task, channel, and model decisions.
1.0	13 Nov 2026	First complete draft for review. Supersedes the phase notes.

ii Contents

1	Executive summary	4
2	Product definition	5
	Vision · problem · principles · non-goals · user	
3	Scope and release boundaries	7
4	User journeys	9
5	System architecture	10
6	Privacy and security model	12
7	Connectors and channels	15
8	Memory, life model, and Galaxy	18
9	Tasks and automation	20
10	Subagents and model lanes	22
11	Interfaces and surfaces	25
12	Data model	26
13	Functional requirements	28
14	Non-functional requirements	32
15	Roadmap and milestones	33
16	Risks and mitigations	35
17	Acceptance criteria and metrics	37
18	Decisions log and open questions	38
A	Glossary	40
B	Configuration reference	41
C	Phase history	43

Reading paths

IF YOU HAVE…	READ
Five minutes	Section 1, Section 3, and the milestone plan in Section 15.
An hour	Sections 1–9, then the decisions log in Section 18. For privacy, add Sections 6–7 and scenarios A9–A10.
To implement	Sections 5–13 and the phase exits in Section 15.2.

1 Executive summary

A.S.H.R.A is a single-user, local-first assistant for Windows 11. It turns a stream of email, calendar events, tasks, and conversations into a small number of clear actions — and asks before it does anything that leaves the machine.

The project began as a 3D knowledge galaxy over markdown notes and a chat box. That foundation is complete and now serves as one surface of a larger product. The next release turns it into an assistant with a life model: who you know, what you owe, what is due, what is waiting, and what it has already done.

Version 1.0 is deliberately narrow. It reads Gmail over IMAP, reads Google Calendar through a private iCal address, keeps a local task ledger, drafts replies, produces a daily brief, and communicates through a Telegram bot with approval buttons. It does not send email, modify a calendar, move money, or touch health data without explicit, per-action approval. It does not read your personal Telegram chats, and it cannot place phone calls.

1	6	3	0
USER, 1 MACHINE	CORE SUBAGENTS	MODEL LANES	SECRETS IN GIT

The problem

Personal administration is not hard because any single task is hard. It is hard because the inputs are scattered and the cost of switching is high. An ask arrives in email, a deadline sits in a calendar invite, a promise is made in a chat, and a bill is due on a date nobody wrote down. Existing assistants solve this by sending everything to a cloud service, which is exactly the trade the owner does not want to make.

The product bet

A small local model, paired with deterministic pipelines, is good enough for triage, extraction, summarisation, and short drafts. Complex planning and tool orchestration should not be delegated to a 3B–4B model. Instead, the system keeps a structured life model, uses rules for anything that must be exact, and uses the local model only where language understanding is genuinely required. Cloud models remain available for non-sensitive work.

> **CORE PROMISE**
>
> **Nothing private leaves this machine, and nothing irreversible happens without a decision.** Every read, draft, approval, and action is attributable, reviewable, and reversible where the underlying system allows it.

What v1.0 delivers

INBOX AWARENESS

Read-only Gmail ingestion with thread reconstruction, local classification, ask extraction, and proposed tasks. Bodies stay on disk.

CALENDAR AWARENESS

Private iCal ingestion, recurrence expansion, conflict detection, and free-slot search for scheduling suggestions.

DAILY BRIEF AND FOLLOW-UPS

A morning brief with today's events, three must-dos, waiting-on items, and one thing to ignore. Unanswered commitments are tracked.

DRAFTS WITH APPROVAL

Replies drafted in your voice, stored in a local outbox, and sent only after an explicit approval from the Desk or Telegram.

PHONE CONTROL

A Telegram bot for briefs, commands, and one-tap approvals, restricted to an allowlisted chat with expiring nonces.

LIFE-MODEL GALAXY

A world-model explorer over memories, people, projects, tasks, events, and sources. Works with zero notes.

What v1.0 withholds

Sending to new recipients, deleting mail, changing calendars, payments, purchases, health actions, smart-home control, arbitrary shell, and any reading of personal chat accounts are out of scope. So is multi-user support, macOS, Linux, and mobile native apps. The architecture leaves room for them; the release does not pretend to include them.

Delivery shape

Work proceeds in small, reviewable phases on named branches, each squash-merged after review. The immediate next step is the connector foundation, which requires no credentials and no model, followed by the local model router so that privacy rules are enforced before any real email is processed.

SECTION 2 PRODUCT DEFINITION

2 Product definition

What A.S.H.R.A is, what it refuses to be, and the principles that decide close calls.

2.1 Vision

A calm, private assistant that holds the administrative shape of a life — commitments, deadlines, people, and open loops — and reduces it to a short, honest answer: *here is what needs you, and here is what I have already handled.*

2.2 Problem statement

The owner runs a working life across email, a calendar, a browser, local files, and chat. The information needed to act is present but fragmented, and each fragment has a different interface. The result is recurring low-grade overhead: re-reading threads to find the ask, remembering promises made in passing, noticing a deadline only when it is close, and rebuilding context before every meeting.

Generic assistants reduce that overhead by centralising content in a vendor cloud. That solves the fragmentation problem by creating a privacy problem. A.S.H.R.A takes the opposite constraint as its starting point: **the machine is the trust boundary**. It must be useful without uploading mail, attachments, calendar detail, or contacts.

2.3 Design principles

#	PRINCIPLE	CONSEQUENCE
P1	**Local is the default, not a mode.**	Private content is processed locally. Cloud calls are an explicit exception with a visible reason.
P2	**Deterministic where correctness matters.**	Dates, recurrence, dedupe, routing, and permissions are code, not model output.
P3	**Propose, then act.**	Every external write has a reviewable proposal and an approval record.
P4	**Structure over screenshots.**	Prefer APIs, IMAP, iCal, and files. Use browser automation only when no structured path exists.
P5	**One brain, many workers.**	One orchestrator owns conversation and approval. Subagents are scoped and cannot act alone.
P6	**Show your evidence.**	Every claim links to the memory, message, or event that supports it.
P7	**Degrade honestly.**	No key, no model, no network, or no connector must produce a clear limitation, never a fabricated answer.
P8	**Small surface, sharp edges.**	Fewer tools, each with a narrow scope, an explicit schema, and a known blast radius.

2.4 Non-goals

- **Not a general chatbot.** Chat is one interface to the life model, not the product's centre.
- **Not a multi-user or team product.** One owner, one machine, one identity. No sharing, no tenancy, no admin console.
- **Not a cloud sync service.** There is no A.S.H.R.A account and no vendor-side storage of private content.
- **Not an autonomous agent that acts unattended.** Unattended work is limited to reads, local writes, and notifications to the owner.
- **Not a financial or medical advisor.** Financial and health data are excluded from automation and model reasoning by policy.
- **Not a replacement for a calendar or mail client.** It reads, summarises, drafts, and proposes; the native clients remain authoritative.

2.5 The user

PRIMARY USER — THE OWNER

One technical Windows user who works across email, a browser, documents, and code. Comfortable with a terminal and a config file. Values privacy over convenience, but will not tolerate friction that repeats every day.

SECONDARY USERS — NONE

People who email or message the owner interact with drafts and replies, never with the assistant directly. They are never asked to install anything or create an account.

2.6 Jobs to be done

WHEN…	I WANT TO…	SO THAT…
I open my laptop	see the shape of the day in under thirty seconds	I start on the right thing instead of the newest thing.
an email arrives with an ask	have it turned into a task or a draft	nothing is lost in the inbox and I do not re-read the thread.
I promise something in passing	have it captured without ceremony	the promise does not quietly expire.
I am away from my desk	approve or reject a draft from my phone	work does not stall on my location.
I ask what I know about a person	see the threads, promises, and tasks in one place	I can prepare for a conversation quickly.
a routine repeats	have it run the same safe way every time	I stop spending attention on it.

> **THE ONE-SENTENCE TEST**
>
> If the assistant cannot explain, in one line, *what it did, on whose behalf, with what evidence, and under which approval*, the feature is not ready.

SECTION 3 SCOPE AND RELEASE BOUNDARIES

3 Scope and release boundaries

v1.0 is a daily-use assistant, not a platform. The boundary is drawn so that everything inside it can be trusted.

3.1 In scope for v1.0

AREA	INCLUDED	PRIORITY
Email	Gmail over IMAP with an app password; read-only sync; thread reconstruction; classification; ask extraction; local drafts; approved SMTP send.	MUST
Calendar	Google Calendar read-only via private iCal URL; recurrence expansion; conflicts; free slots; drafted `.ics` invites.	MUST

AREA	INCLUDED	PRIORITY
Tasks	Local ledger with inbox, next, waiting, scheduled, done; due dates; reminders; recurrence; dedupe; source links.	MUST
Briefs	Morning and evening briefs; waiting-on list; follow-up detection; weekly review.	MUST
Telegram	Allowlisted bot; text commands; briefs; approval buttons; quiet hours; audit of outbound messages.	MUST
Desk	Run timeline, approvals queue, outbox, audit log, connector status.	MUST
Memory	Typed memories with provenance, sensitivity, and review; capture proposals; confirmation loop.	MUST
Galaxy	Runtime world-model graph over memories, people, projects, tasks, events, sources, connectors.	SHOULD
Files and apps	Allowlisted folders; dry-run preview; approved move, rename, delete-to-recycle; app launch routines.	SHOULD
Browser	Playwright with a dedicated profile; allowlisted domains; research; form fill with approval before submit.	COULD
Scripts	Manifested local scripts with argument schemas and per-run approval.	COULD

3.2 Explicitly out of scope

NEVER IN ANY RELEASE

- Payment initiation or banking actions
- Health, medication, or medical advice actions
- Reading personal Telegram chats via a userbot
- Arbitrary shell execution from model output
- Sending private content to a cloud model

NOT IN V1.0

- Sending to new recipients without approval
- Calendar writes via OAuth
- Voice calls and SMS
- Smart-home control
- Mobile native apps
- macOS or Linux packaging

LATER, IF JUSTIFIED

- Twilio voice and SMS
- Slack and Teams connectors
- Calendar write via OAuth
- Local speech-to-text and TTS
- Remote access beyond the LAN
- Multi-device sync

3.3 Release definition

v1.0 is reached when the owner can complete a normal working day using A.S.H.R.A as the first screen: read the brief, work the queue, approve or reject drafts, capture tasks by voice or text, and close the day with a review — without any private content leaving the machine and without a single unapproved external write.

> **SCOPE DISCIPLINE**
>
> A feature that cannot be described as a read, a local draft, an approved action, or a notification is probably a new product. Park it in Section 18 rather than stretching v1.0.

4 User journeys

Six journeys define the daily product. Each one states what the user does, what the system does, and where approval sits.

4.1 Morning brief

STEP	ACTOR	BEHAVIOUR
1	Scheduler	At the configured wake time, builds a brief from today's events, tasks, waiting-on items, and overnight email.
2	Local model	Ranks at most three must-dos and writes one line of context for each. No email body is quoted.
3	Telegram	Delivers the brief with buttons: *Open Desk*, *Snooze 1h*, *Quiet today*.
4	User	Reads on phone, taps through to the Desk for detail, or replies with a command.

Approval: none required. The brief is outbound to the owner only and contains summaries, never bodies or attachments.

4.2 Inbox triage to draft

STEP	ACTOR	BEHAVIOUR
1	Connector	Fetches new mail with `BODY.PEEK`, so nothing is marked read. Stores headers and bodies locally.
2	Pipeline	Reconstructs threads, strips signatures and quotes, detects newsletters and receipts by rule.
3	`inbox_triager`	Classifies each thread: reply needed, waiting, informational, or ignore. Extracts asks, people, and dates.
4	System	Proposes tasks and memories. Creates nothing without confirmation, except explicit user requests.
5	`writer`	On request or by rule, drafts a reply in the owner's voice. Saves it to the local outbox.
6	User	Reviews in the Desk, edits, then approves. Send is a separate, explicit action.

4.3 Promise capture

The user says or types: *"Remember that I promised Sarah the deck by Tuesday."* The assistant creates a task with `person = Sarah`, `due = Tuesday`, `status = next`, links the source message, and confirms in one line: *"Captured: send Sarah the deck, due Tue 17 Nov."* No further dialogue. If the date is ambiguous, it asks exactly one clarifying question.

4.4 Phone approval

STEP	ACTOR	BEHAVIOUR
1	System	Posts a draft summary to Telegram with *Approve*, *Edit in Desk*, *Reject*. The message carries a one-time nonce valid for 24 hours.
2	User	Taps *Approve*.
3	Approval service	Validates chat allowlist, nonce, expiry, and current draft hash. Rejects stale or replayed taps.
4	Operator	Sends through SMTP. Writes an audit record with recipient count, draft hash, and approval id.
5	Telegram	Confirms: *"Sent to Sarah (1 recipient)."* The button is removed.

4.5 Scheduling a meeting

The user asks for thirty minutes with Sarah next week. `calendar_scout` reads the local calendar cache, finds three candidate slots that avoid conflicts and respect working hours, and `scheduler` drafts an invite as an `.ics` file plus an email draft. Nothing is sent and no event is created. The user approves, then either sends the draft or imports the `.ics` into Google Calendar.

4.6 File cleanup and app routines

STEP	ACTOR	BEHAVIOUR
1	User	"Clean up Downloads."
2	`file_operator`	Scans an allowlisted root, groups by type and age, and produces a dry-run plan with counts and total size.
3	Desk	Shows every move and rename. Deletions go to the Recycle Bin, never permanent.
4	User	Approves the batch once. The operator executes and records each operation.
5	System	Offers to save the batch as a reusable routine with its own scope and approval policy.

4.7 End of day

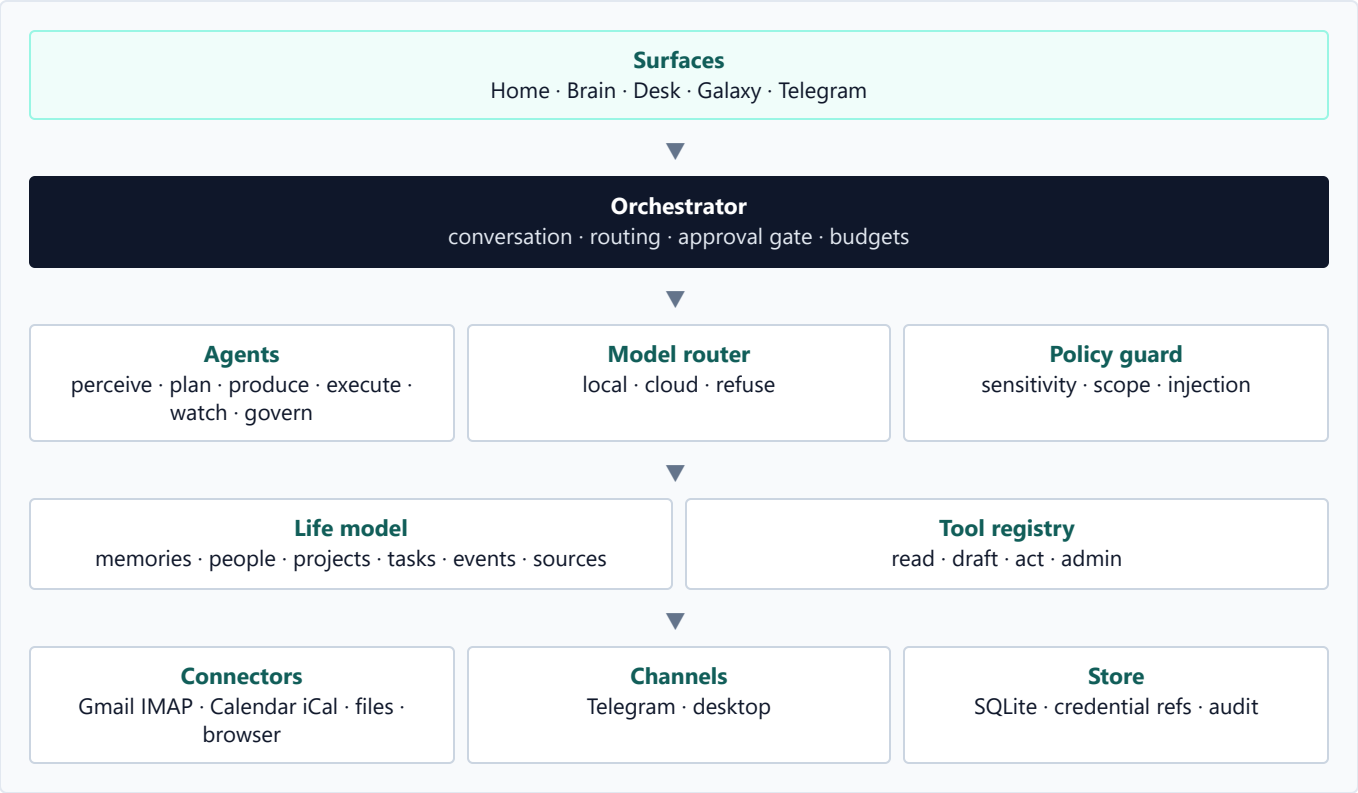
An evening brief summarises what moved, what is still waiting, and what tomorrow looks like. It proposes up to three tasks for tomorrow and asks one question: *"Anything to add before I close the day?"* The answer is captured as a task or memory, and quiet hours begin.

SECTION 5 SYSTEM ARCHITECTURE

5 System architecture

One FastAPI process, one SQLite database, one local model, and a set of narrow connectors. Everything private stays inside this boundary.

5.1 Layered view



F-1 — Layered system architecture. Arrows show the only permitted direction of calls; lower layers never call upward.

5.2 Components

COMPONENT	RESPONSIBILITY	MUST NOT
`api`	HTTP routes, request validation, static surfaces, SSE streaming.	Hold business logic or secrets in responses.
`orchestrator`	Owns the turn: plan, dispatch, collect proposals, request approval, record the run.	Let a subagent act without an approval record.
`model_router`	Selects local, cloud, or refusal by sensitivity and task type.	Fall back to cloud when the local model fails.
`policy_guard`	Checks sensitivity, tool scope, grant validity, and injection signals.	Be bypassable by prompt text or tool arguments.
`memory`	Typed memories, retrieval, graph edges, sensitivity, lifecycle.	Store a memory without provenance.
`tasks`	Local ledger, status transitions, recurrence, dedupe, reminders.	Create an extracted task without confirmation.
`connectors/*`	One adapter per external system, with a capability declaration.	Exceed its declared read/write capability.
`channels/*`	Outbound delivery, inbound commands, approval UI.	Send private bodies or accept unknown identities.

COMPONENT	RESPONSIBILITY	MUST NOT
`credentials`	Windows Credential Manager wrapper; returns opaque refs only.	Write secrets to config, SQLite, logs, or git.
`audit`	Append-only record of reads, drafts, approvals, actions, and outbound messages.	Store message bodies or attachment contents.

5.3 Turn lifecycle

1. **Intake.** A surface submits a request with an identity and a surface tag. Telegram requests must match the allowlisted chat.
2. **Classify.** The router determines intent deterministically where possible. Only ambiguous intent reaches a model.
3. **Scope.** The policy guard computes the maximum permission and the permitted model lane for this turn.
4. **Gather.** Read tools and agents collect evidence. Every item carries a source reference and sensitivity.
5. **Produce.** A produce agent writes text or a proposal. Output is validated against a schema before it is shown.
6. **Decide.** If the turn implies an external write, the orchestrator creates an approval request with a payload hash.
7. **Act.** On approval, a domain operator executes exactly the approved payload. The payload is immutable after approval.
8. **Record.** The run, tokens, cost, evidence ids, approval id, and result are written to the audit log.

5.4 Deployment shape

```
Windows 11 host
├─ python server.py            FastAPI on 127.0.0.1:4700
├─ ollama serve                local model on 127.0.0.1:11434
├─ data/ashra.db               SQLite: sessions, tasks, memories, audit
├─ data/cache/                 email and calendar cache (encrypted at rest)
├─ data/outbox/                drafts and generated .ics files
├─ config.json                 non-secret configuration only
└─ Windows Credential Manager  IMAP password, iCal URL, Telegram token
```

The server binds to loopback only. There is no inbound port, no tunnel, and no public URL in v1.0. Telegram uses long polling, so the assistant reaches out rather than listening.

> **ARCHITECTURAL INVARIANT**
>
> The orchestrator is the only component allowed to request approval, and domain operators are the only components allowed to execute external writes. No agent holds both capabilities.

SECTION 6 PRIVACY AND SECURITY MODEL

6 Privacy and security model

Privacy is not a setting. It is a set of classifications and gates that every request passes through before it reaches a model, a channel, or a tool.

6.1 Sensitivity classes

CLASS	EXAMPLES	MODEL	CHANNEL
PUBLIC	Web pages, public documentation, general knowledge.	Cloud or local	Allowed
INTERNAL	Personal notes, project plans, non-sensitive research.	Cloud or local	Summaries
PRIVATE	Email bodies, calendar details, contacts, drafts, tasks.	**Local only**	Counts and summaries
SENSITIVE	Attachments, identity documents, private conversations, credentials metadata.	**Local only**	Never
SECRET	Financial records, health information, passwords, tokens.	**No model**	Never

Classification is assigned at ingestion, inherited by derived records, and never downgraded automatically. A mixed request takes the highest class present. If classification is uncertain, the system chooses the more private class and says so.

6.2 Model lanes

LOCAL LANE

- 3B–4B model through Ollama
- Email, calendar, contacts, tasks
- Classification, extraction, summaries, short drafts
- No network egress

CLOUD LANE

- Configured provider, OpenAI-compatible
- Public and non-sensitive internal content
- Research, general planning, long-form writing
- Explicit reason recorded per call

REFUSAL LANE

- Secret-class content
- No model of any kind
- Rules, search, and reminders only
- Explains the policy to the user

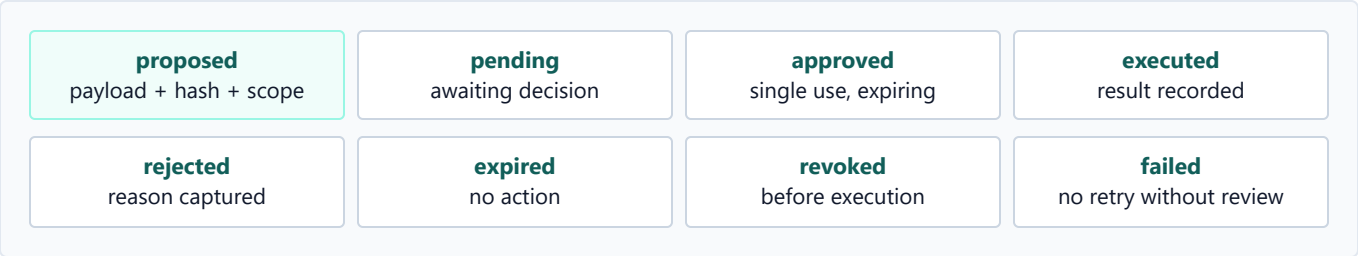
F-3 — Model lane routing. The router may downgrade to local or refuse; it may never upgrade to cloud on failure.

6.3 Credentials

SECRET	STORAGE	USED BY
Gmail app password	Windows Credential Manager, target `ashra/gmail`	IMAP and SMTP connector
Google Calendar iCal URL	Windows Credential Manager, target `ashra/calendar`	Calendar connector
Telegram bot token	Windows Credential Manager, target `ashra/telegram`	Telegram channel
Cloud model API key	Windows Credential Manager or `config.json` as today	Cloud provider only

SQLite stores a credential reference, never a value. Logs redact values by pattern and by key name. The repository ignores `config.json`, `data/`, and any file matching `*.secret`. A pre-commit check rejects high-entropy strings in tracked files.

6.4 Approval state machine



F-4 — Approval state machine. Approved payloads are immutable; any edit returns the request to *proposed*.

An approval binds four things: the exact payload, the tool and its scope, the requesting surface, and an expiry. Telegram approvals also bind the chat id and a one-time nonce. Replays, stale hashes, and edits after approval are rejected and audited.

6.5 Prompt-injection defence

- Email, calendar, web, and document content is **data, not instruction**. It is wrapped in an untrusted-content envelope before it reaches a model.
- The policy guard scans for directive patterns — "ignore previous instructions", "send this to", "run this command" — and raises the risk score of the turn.
- Tool selection is deterministic. A model may propose an action, but it cannot add tools, widen scope, or change recipients.
- Links in messages are never fetched automatically. Fetching is a separate, visible step.
- Any request in content to install software, send credentials, or contact a third party is surfaced to the user as a suspected injection.

6.6 Data retention

DATA	RETENTION	CONTROL
Email headers and bodies	90 days by default, configurable	Purge command; per-thread delete
Attachments	Cached only while a task references them	Never parsed by cloud OCR
Calendar cache	Rolling 60 days back, 180 forward	Rebuilt on demand
Memories	Until forgotten	Per-item forget; source link retained in audit only
Audit events	1 year	Metadata only; exportable
Drafts and outbox	Until sent or discarded	Manual purge

6.7 Threat model

THREAT	IMPACT	MITIGATION
Malicious email content	Injection, data exfiltration	Untrusted envelope, deterministic tools, no auto-fetch, approval gate.

THREAT	IMPACT	MITIGATION
Stolen laptop	Mail and calendar exposure	Windows account protection, encrypted cache, credential vault, short retention.
Leaked bot token	Unauthorised channel access	Allowlisted chat id, token rotation, no private content in messages.
Cloud provider logging	Private content retention	Private content never routed to cloud; calls logged locally by hash.
Runaway agent loop	Cost and unintended action	Depth limit 2, token and time budgets, no subagent writes.
Accidental send	Reputational harm	Separate send step, recipient review, undo window where the provider allows.

HARD RULE

If a request mixes secret-class content with anything else, the whole turn is refused. The assistant may say that it cannot help with that content and offer a manual path. It may not summarise, redact, or partially process it.

SECTION 7 CONNECTORS AND CHANNELS

7 Connectors and channels

A connector reads or writes one external system. A channel carries messages to or from the owner. Both declare their capabilities up front, and neither may exceed them.

7.1 Connector contract

```
Connector
  id            "gmail" | "gcal" | "files" | "browser"
  capabilities  { read, draft, write, delete }  // declared, enforced
  scopes        folders, domains, calendars, roots
  auth          credential_ref -> Windows Credential Manager
  poll()        incremental fetch, idempotent by external id
  normalise()   external record -> Source + Event + candidate Task
  health()      last success, last error, backlog, latency
  redact()      strip secrets and sensitive fields before logging
```

Every connector is idempotent, resumable, and rate-limited. Failures degrade to a visible status in the Desk; they never silently disable a capability.

7.2 Email — Gmail over IMAP

TRANSPORT

IMAP over TLS with a Gmail app password stored in the credential vault. SMTP over TLS is used only in the approved-send phase.

READ-ONLY FIRST

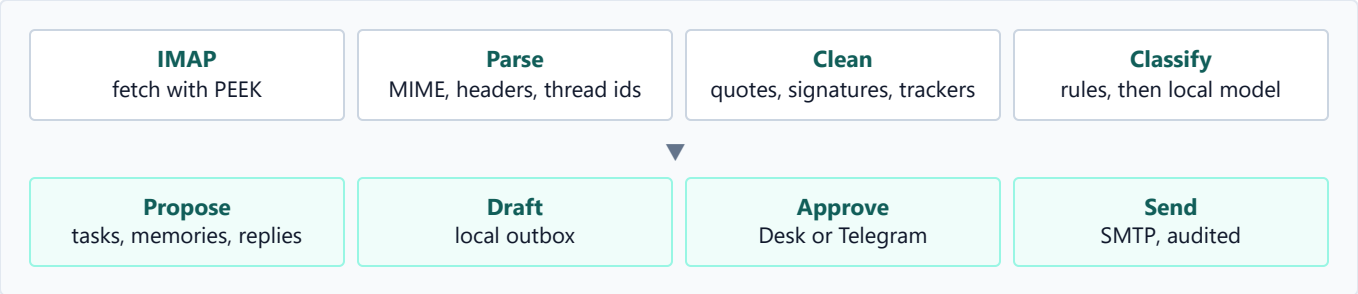
Fetch uses `BODY.PEEK`. No `STORE`, `APPEND`, `EXPUNGE`, label change, or move in the read phase.

SYNC MODEL

UIDVALIDITY plus UID cursor, stored per folder. A resync is a full pass; normal operation is incremental.

SCOPE

Inbox, plus an optional allowlist of labels. Sent mail is read only to learn voice and thread context, never quoted outward.



F-2 — Email read-and-draft pipeline. Every stage after *Classify* produces a proposal, never an external write.

MESSAGE HANDLING RULES

- Newsletters, receipts, and automated notifications are classified by deterministic rules, not by the model, and are excluded from the must-reply queue.
- Attachments are stored locally with a content hash and never sent to a cloud service. Only text extracted locally may inform a draft.
- Quoted history is collapsed before model input to reduce tokens and accidental disclosure.
- Drafts are stored as structured records — recipient, subject, body, in-reply-to, attachments — so a send is reproducible and auditable.
- An approved send binds the exact draft hash. Editing the draft invalidates the approval.

AUTHENTICATION CAVEAT

Gmail app passwords remain supported but are a legacy path. If Google retires them for this account, the fallback is Gmail API OAuth with restricted scopes. In OAuth Testing mode, refresh tokens expire about every seven days, which is why IMAP is preferred for v1.0. The connector interface isolates this change to one adapter.

7.3 Calendar — Google Calendar via private iCal

The connector fetches the private iCal address for each selected calendar and parses it locally. It expands recurrence rules, resolves time zones, and builds a rolling window of events. It never writes to Google Calendar in v1.0.

CAPABILITY	V1.0	NOTES
Read events	YES	Private iCal URL, 60 days back and 180 forward.
Recurrence	YES	RRULE, EXDATE, and time-zone expansion in local code.
Free/busy search	YES	Computed from the local cache, respecting working hours.
Draft invite	YES	Generated `.ics` file the user can review and import.
Create or edit event	NO	Requires OAuth write scope; deferred.

7.4 Telegram — control channel

Telegram is the first channel because it is free, reliable on a phone, supports buttons and voice notes, and works with long polling so the local server never exposes a port. It is a control surface, not a data store.

COMMAND	EFFECT
`/brief`	Send today's brief now.
`/tasks`	List open tasks, grouped by status.
`/task buy batteries tomorrow`	Create a task with parsed due date and confirm.
`/approve <id>` or button	Approve a pending action, subject to nonce and expiry.
`/deny <id>` or button	Reject with an optional reason.
`/mute 2h`	Enter quiet hours immediately.
`/status`	Connector health, queue depth, model lane, last sync.

CHANNEL RULES

- Only the allowlisted chat id is accepted. Unknown chats are ignored and logged, never answered.
- Approval buttons carry a nonce, an expiry, and the payload hash. A button can be used once.
- Messages contain counts, times, and summaries. Email bodies, attachments, calendar titles with attendees, and contact details stay in the Desk.
- Rate limits, deduplication, and quiet hours apply to every outbound message.
- Every outbound message is recorded in the audit log with its template id and approval id, if any.

> **LIMITS TO STATE PLAINLY**
>
> A Telegram bot cannot read personal chats it is not part of, cannot place a voice call, and cannot send SMS. Real calls and SMS require a telephony provider such as Twilio and are deferred. Voice notes are a later addition with local speech-to-text.

7.5 Files and applications

The file operator works only inside declared roots — by default `Downloads` and a staging folder. It produces a dry-run plan before any change, moves deletions to the Recycle Bin, and records every operation. The app operator launches allowlisted executables and can arrange windows; it never types into an application unless a specific, approved routine declares the target and the text.

7.6 Browser

Browser automation uses Playwright with a dedicated profile, an allowlist of domains, and a visible session. It is used for research and for forms that have no API. It may fill fields and prepare a submission, but a submit, purchase, booking, or upload requires approval. Banking, health, and payment domains are blocked at the policy layer, not by prompt instruction.

7.7 Scripts

A script is registered in a manifest with a name, path, argument schema, timeout, expected output, and risk class. Running it requires approval per run unless a scoped grant exists. The runner captures stdout and stderr, truncates secrets, and records the exit code. Arbitrary shell from model output is never executed.

7.8 Capability matrix

CONNECTOR	READ	DRAFT	WRITE	DELETE	APPROVAL REQUIRED FOR
Gmail IMAP	Yes	Local	Send	No	Any send; any recipient change
Google Calendar	Yes	.ics	No	No	Nothing in v1.0
Telegram	Commands	—	Messages	—	Outbound to anyone but the owner
Files	Yes	Plan	Move	Recycle	Any batch change
Browser	Yes	Fill	Submit	No	Submit, upload, purchase
Scripts	—	—	Run	—	Every run unless granted

SECTION 8 MEMORY, LIFE MODEL, AND GALAXY

8 Memory, life model, and Galaxy

Notes are a source. The life model is the product. The Galaxy is how you see it.

8.1 From notes to life model

The current build treats markdown notes as the centre of retrieval. That inverts the real dependency. Notes are one input among many, and the test notes currently in the repository are not a foundation. The life model must be useful with zero notes.

ENTITY	REPRESENTS	CREATED BY
Memory	A durable fact, preference, decision, or observation with provenance.	Explicit capture, confirmed proposal, import.
Person	Someone you correspond or work with; last contact, open commitments.	Email and calendar ingestion, manual entry.
Project	A body of work with a status, owner, and related items.	Manual, or promoted from repeated tasks.
Task	A unit of intended work with status, due date, and source.	Explicit request, confirmed extraction.
Event	A calendar item, deadline, or scheduled block.	Calendar connector, task due date.
Source	The email, message, note, document, or meeting an item came from.	Connectors.
Connector	A configured external system and its health.	Setup.

8.2 Memory lifecycle

1. **Propose.** A capture proposal is created with text, type, sensitivity, confidence, and at least one source reference.
2. **Confirm.** The user accepts, edits, or rejects. An explicit "remember that…" may confirm immediately.
3. **Store.** The memory is written with an immutable id, a timestamp, and a provenance edge.
4. **Retrieve.** Hybrid search: keyword, recency, type filter, and graph proximity. Sensitivity filters apply before ranking.
5. **Supersede.** A newer memory can replace an older one without deleting history. Both remain auditable.
6. **Forget.** The user can forget a memory; its source link is removed and the audit records the action, not the content.

> **CAPTURE POSTURE**
>
> The assistant proposes; the user confirms. The only shortcut is an explicit instruction to remember, which is treated as confirmation. Nothing is learned silently from email.

8.3 Galaxy as world-model explorer

The Galaxy stops being a map of markdown files and becomes a map of what the assistant knows and owes. It is a sense-making tool, not a task board. Dense task work belongs in Home and the Desk.

MODE	QUESTION IT ANSWERS	PRIMARY NODES
Explore	What do I know about this person or project?	Memory, Person, Project, Source
Today	What is relevant right now?	Event, Task, open loop
Open loops	What is waiting on me or someone else?	Task, commitment, thread

MODE	QUESTION IT ANSWERS	PRIMARY NODES
People	Who do I owe, and who owes me?	Person, commitment
Provenance	Why do you believe this?	Memory → Source
Recent	What changed this week?	Capture, run, approval, connector

8.4 Graph contract

```
GET /world/graph?mode=today&depth=2

{
  "nodes": [
    {"id": "mem_...", "type": "memory", "label": "...",
     "sensitivity": "private", "weight": 0.8}
  ],
  "edges": [
    {"from": "mem_...", "to": "per_...", "type": "about"}
  ],
  "meta": {"mode": "today", "truncated": false, "generated_at": "..."}
}
```

Edge types: `mentions`, `about`, `owns`, `works_with`, `due_on`, `part_of`, `replied_to`, `derived_from`, `blocked_by`, `promised_to`. The graph is generated at request time from SQLite, cached briefly, and bounded by node and edge limits so a large mailbox cannot freeze the view.

8.5 Notes as an optional connector

The markdown vault remains supported: `build.py` can still index notes, and useful notes can be promoted to document memories with a source edge. If the vault is deleted, nothing breaks. The Galaxy falls back to memories, people, projects, tasks, events, and connectors. This is a deliberate decoupling, not a migration deadline.

SECTION 9 TASKS AND AUTOMATION

9 Tasks and automation

Tasks are the unit of accountability. Automation is a ladder, and each rung has a different authority.

9.1 Task model

FIELD	TYPE	RULES
`id`	string	Stable, prefixed `tsk_`.
`title`	string	Imperative, under 120 characters, no trailing period.
`notes`	text	Optional detail; may quote a source locally.

FIELD	TYPE	RULES
`status`	enum	`inbox`, `next`, `waiting`, `scheduled`, `done`, `cancelled`.
`priority`	enum	`low`, `normal`, `high`. No more than three high tasks per day.
`due_at`, `remind_at`	datetime	Local time zone; reminders default to 09:00 on the due date.
`project`, `person`	ref	Optional links to the life model.
`source`, `source_ref`	ref	Required for extracted tasks; null for manual capture.
`estimate_min`, `energy`	int, enum	Used by the planner to fit the day.
`recurrence`	rule	Subset of RFC 5545; expanded locally.
`parent_id`	ref	One level of subtasks, no deeper.

9.2 Creation policy

SOURCE	BEHAVIOUR	CONFIRMATION
Explicit request — "remind me to…"	Create immediately.	One-line confirmation.
Email or message extraction	Create a proposal in the queue.	Required before it becomes a task.
Recurring routine	Create from a template.	Once, at template creation.
Brief suggestion	Propose up to three for tomorrow.	Required; accept all or individually.

Deduplication uses a normalised title, the linked source, and a due-date window. If a similar open task exists, the assistant offers to update it instead of creating a second one.

9.3 The autonomy ladder

LEVEL	EXAMPLE	AUTHORITY	APPROVAL
0 · Remind	"Renew insurance Friday."	Local task and notification.	None
1 · Draft	"Draft a reply to Sarah."	`DRAFT`; stored locally.	None
2 · Prepare	"Find 30 minutes with Sarah."	Slots and `.ics` prepared.	None
3 · Act	"Send it." "Archive these."	`ACT` on the exact payload.	Per action
4 · Grant	"Archive newsletters automatically."	Scoped, expiring, audited.	Once, at grant

T-1 — Autonomy ladder. v1.0 ships levels 0–3. Level 4 exists in the permission model but starts with no grants.

9.4 Reminders and recurrence

Reminders are generated locally and delivered through the active channel. Quiet hours suppress delivery until the next allowed window, except for a task explicitly marked urgent. Recurrence is expanded in code, not by the model, and completed instances are recorded so that a recurring task never double-fires.

9.5 Daily brief

The brief is a fixed template with variable content. It contains today's events with times, at most three must-dos, a waiting-on list, one item to ignore, and a single question. It is generated from structured data first; the local model only writes connective prose. If the model is unavailable, the brief still renders.

9.6 What is never automated

MONEY
No payments, transfers, purchases, or trading. Due-date reminders only.

HEALTH
No medication, appointment, or medical-record actions. Reminders only, with no model involvement.

IRREVERSIBLE DESTRUCTION
Deletions go to the Recycle Bin. Permanent deletion requires a separate, explicit confirmation.

IDENTITY AND ACCESS
No password resets, account changes, or credential sharing.

AUTOMATION TEST

Before adding an automation, answer: what is the worst outcome if it runs while I am asleep? If the answer involves money, health, a third party, or permanent data loss, it is not an automation — it is a proposal.

SECTION 10 SUBAGENTS AND MODEL LANES

10 Subagents and model lanes

Six families answer six different questions. An agent is added only when a workflow repeats, and every agent runs inside the parent's authority.

10.1 Families

FAMILY	QUESTION	CEILING	OUTPUT
Perceive	What is true right now?	`READ`	Evidence, extracted facts, candidate tasks.
Plan	What matters, in what order?	`READ` / `DRAFT`	Plan, ranked list, risks, dependencies.
Produce	What should we say or make?	`DRAFT`	Draft, summary, invite, document.
Execute	Do the approved thing.	`ACT`	Action result and audit record.
Watch	What changed, and what is next?	`READ`	Alert, brief, proposal.

FAMILY	QUESTION	CEILING	OUTPUT
Govern	Is this safe and correct?	System	Routing, veto, quality score.

10.2 Roster

AGENT	FAMILY	PERMISSION	LANE	PURPOSE
`researcher`	Perceive	READ	EITHER	Reads memory, documents, and the web; returns evidence, not opinion.
`inbox_triager`	Perceive	READ	LOCAL	Classifies threads; extracts asks, people, and dates.
`calendar_scout`	Perceive	READ	LOCAL	Finds free slots, conflicts, prep work, and travel gaps.
`document_reader`	Perceive	READ	LOCAL	Extracts obligations and dates from PDFs and attachments.
`contact_analyst`	Perceive	READ	LOCAL	Builds relationship context from mail and events.
`planner`	Plan	READ	EITHER	Turns goals and constraints into a realistic day or week.
`critic`	Plan	READ	EITHER	Checks plans and drafts for contradictions and risk.
`writer`	Produce	DRAFT	LOCAL	Drafts in the owner's voice; local for mail, cloud for public content.
`summarizer`	Produce	DRAFT	EITHER	Condenses threads and meetings; local when content is private.
`scheduler`	Produce	DRAFT	LOCAL	Proposes times and drafts invites; never sends.
`mail_operator`	Execute	ACT	LOCAL	Sends or archives exactly the approved payload.
`calendar_operator`	Execute	ACT	LOCAL	Writes calendar changes when OAuth write exists; deferred.
`task_manager`	Execute	ACT	LOCAL	Creates and updates the local ledger.
`file_operator`	Execute	ACT	LOCAL	Runs approved batches inside allowlisted roots.
`browser_operator`	Execute	ACT	EITHER	Allowlisted research and form preparation.
`notifier`	Execute	ACT	LOCAL	Delivers summaries to the owner; approval for anyone else.
`watchkeeper`	Watch	READ	LOCAL	Watches rules, deadlines, and anomalies; proposes only.
`followup_tracker`	Watch	READ	LOCAL	Tracks unanswered mail, promises, and stalled threads.
`briefing_agent`	Watch	READ	LOCAL	Builds morning, evening, and weekly briefs.

AGENT	FAMILY	PERMISSION	LANE	PURPOSE
`router`	Govern	System	Det.	Intent routing; the model is used only when ambiguous.
`policy_guard`	Govern	System	Det.	Scope, grants, sensitivity, injection; may veto.
`evaluator`	Govern	System	EITHER	Scores output quality and records metrics.

T-2 — Subagent roster. The core six are `researcher`, `inbox_triager`, `planner`, `writer`, `mail_operator`, and `watchkeeper`.

10.3 Agent contract

```
AgentSpec
  id, family, permission_ceiling, lane, tools[]
  input_schema, output_schema
  max_tokens, max_seconds, max_tool_calls
  memory_policy: "read" | "propose" | "none"
  escalation: return_to_orchestrator

AgentResult
  answer, evidence[], proposed_tasks[], proposed_memories[],
  proposed_actions[], confidence, cost, limitations[]
```

Agents return structured proposals. They do not send free-form handoffs to each other, and they cannot widen their own tool list. Effective permission is the intersection of the agent ceiling, the parent's authority, and the current turn's scope.

10.4 Rules

- **One orchestrator.** It owns the conversation, the approval request, and the final answer. Subagents are workers.
- **No irreversible action from a subagent.** Execute agents act only on an approved payload supplied by the orchestrator.
- **Depth limit 2.** An agent may call one level of helper and no further.
- **Budgets.** Per-turn token, time, and tool-call caps. Exceeding a budget returns a partial result with a limitation note.
- **Shared memory.** Only the memory curator proposes new memories. Agents read; they do not write.
- **Add on repetition.** A new agent requires at least three repeated workflows, a distinct output schema, and a permission profile that differs from existing agents.

10.5 Model routing rules

TASK	LANE	FALLBACK
Email classification and extraction	Local	Rules only; no cloud fallback.
Email draft	Local	Template draft; no cloud fallback.
Calendar summary	Local	Deterministic rendering.

TASK	LANE	FALLBACK
General chat	Cloud	Local model with a quality warning.
Public web research	Cloud	Local model on extracted text.
Planning with private context	Local	Rule-based plan from tasks and events.
Secret-class content	None	Refusal with a manual path.

11 Interfaces and surfaces

Five surfaces, each with one job. If a screen tries to do two jobs, it belongs somewhere else.

11.1 Home

The default screen and the daily starting point. It shows the current brief, the **Needs You** queue, today's events, open tasks, and a capture bar. The queue is ordered by consequence and deadline, not by arrival time. Every item can be approved, snoozed, or opened in the Desk.

REGION	CONTENT	ACTIONS
Brief	Three must-dos, waiting-on, one thing to ignore.	Refresh, mute, open Desk.
Needs You	Pending approvals and proposals, with source and expiry.	Approve, reject, edit, snooze.
Today	Events and scheduled tasks on a single timeline.	Open event, draft prep, reschedule task.
Capture	One input for tasks and memories.	Create, confirm, undo.

11.2 Desk

The control hub for work in flight. It contains the run timeline, the approvals queue, the outbox, connector health, and the audit log. It is the only place where a user can see exactly what the assistant did, with what evidence, and under which approval.

- **Runs:** every turn, its agents, tools, tokens, cost, duration, and result.
- **Approvals:** pending, approved, rejected, expired, revoked; payload preview and hash.
- **Outbox:** drafts with recipient, subject, body, and send state.
- **Connectors:** last sync, backlog, error, and a manual resync.
- **Audit:** filterable by actor, surface, tool, and date; export to JSON.

11.3 Brain

The memory surface. It supports capture, review, search, and forgetting. Each memory shows its type, sensitivity, confidence, source, and the edges that connect it to people, projects, and tasks. Superseded memories remain

visible behind a history toggle.

11.4 Galaxy

The world-model explorer described in Section 8. It opens in *Today* mode by default, with *Explore*, *Open loops*, *People*, *Provenance*, and *Recent* available. A node click opens a side panel with evidence and a link to the Desk. It is a read-only view; nothing is edited in the graph.

11.5 Telegram

The away-from-desk surface. It carries briefs, short answers, task capture, and approvals. It never carries full email bodies or attachments. Its keyboard is limited to actions that are safe to take from a phone.

11.6 Surface rules

RULE	RATIONALE
Approval is always one tap away from the item.	Friction causes rubber-stamping; distance causes abandonment.
Every summary links to its evidence.	Trust requires inspectability.
No private body in a notification.	Lock screens and notification previews leak.
Destructive actions are visually distinct and never the default button.	Mis-taps should be harmless.
Empty states explain what to connect.	A blank Galaxy is an onboarding failure, not a design statement.

DESIGN TONE

Quiet, dense, and honest. No celebration animations for routine work, no anthropomorphic flattery, no fake urgency. The interface should feel like a good chief of staff: brief, prepared, and easy to correct.

SECTION 12 DATA MODEL

12 Data model

One SQLite database, explicit tables, no hidden state. Secrets never appear in a row.

12.1 Tables

TABLE	KEY COLUMNS	PURPOSE
`sessions`	id, surface, created_at, title	Conversation grouping across surfaces.
`messages`	id, session_id, role, content, created_at	Turn history. Private content stays local.

TABLE	KEY COLUMNS	PURPOSE
`runs`	id, session_id, intent, agents, tokens, cost, status, started_at	One orchestrated turn, for the Desk timeline.
`approvals`	id, run_id, tool, payload_hash, payload, scope, status, expires_at	Immutable action proposals and decisions.
`audit_events`	id, at, actor, surface, action, target, approval_id, meta	Append-only record. Metadata, never bodies.
`memories`	id, type, text, sensitivity, confidence, source_id, superseded_by	Typed facts with provenance.
`memory_edges`	from_id, to_id, type, weight	Graph relations between life-model entities.
`people`	id, name, emails, last_contact_at, notes_ref	Relationship context.
`projects`	id, name, status, owner, due_at	Grouping for tasks and sources.
`tasks`	id, title, status, priority, due_at, project_id, person_id, source_id, recurrence	Local ledger.
`sources`	id, kind, external_id, thread_id, subject, occurred_at, local_path	Email, message, note, document, meeting.
`events`	id, source_id, start_at, end_at, title, location, attendees	Calendar cache and derived deadlines.
`drafts`	id, source_id, to, subject, body, attachments, state, hash	Outbox. Body local; hash binds approval.
`grants`	id, tool, scope, granted_at, expires_at, revoked_at	Scoped autonomy. Empty at v1.0 launch.
`connectors`	id, kind, credential_ref, scopes, last_sync_at, status	Configuration and health.
`outbound`	id, channel, template, approval_id, sent_at, status	Every message sent to the owner or others.

12.2 Invariants

- Every memory, task, and event has a `source_id` or an explicit `manual` marker.
- Every approval references a run and a payload hash; an executed approval cannot be reused.
- Audit rows are append-only. Updates are modelled as new events.
- Credential columns store references only. A migration test asserts that no column name matches a secret pattern with a non-null value.
- Deleting a source cascades to derived proposals but not to confirmed memories; those are marked `orphaned` and reviewed.

12.3 Example queries the model must support

```
-- Today's commitments, ordered by consequence
SELECT t.title, t.due_at, p.name AS person
FROM tasks t LEFT JOIN people p ON p.id = t.person_id
WHERE t.status IN ('next','scheduled') AND date(t.due_at) <= date('now','localtime')
ORDER BY t.priority DESC, t.due_at ASC LIMIT 20;

-- Open loops for a person
SELECT m.text, m.created_at
FROM memories m JOIN memory_edges e ON e.from_id = m.id
WHERE e.to_id = ? AND e.type = 'promised_to' AND m.superseded_by IS NULL;
```


13 Functional requirements

Each requirement is testable and has one owner area. Priority follows Section 3.1.

13.1 Privacy and policy

ID	REQUIREMENT	PRIORITY
FR-P1	Every ingested record receives a sensitivity class and inherits it to all derived records.	MUST
FR-P2	The model router blocks private and sensitive content from any cloud provider, including retries and fallbacks.	MUST
FR-P3	Secret-class content is never sent to a model and never automated; the user receives a clear refusal and a manual path.	MUST
FR-P4	Credentials are stored in Windows Credential Manager; the database and logs contain references only.	MUST
FR-P5	Untrusted content is enveloped and scanned for directive patterns before model input.	MUST
FR-P6	Every cloud model call records its reason and the sensitivity class of its payload.	SHOULD
FR-P7	A purge command removes cached mail, calendar, and attachment data within one minute.	MUST

13.2 Email

ID	REQUIREMENT	PRIORITY
FR-E1	Connect to Gmail over IMAP using a credential-vault app password, with a visible connection test.	MUST
FR-E2	Fetch new mail incrementally using UID cursors and `BODY.PEEK`, without changing read state.	MUST
FR-E3	Reconstruct threads from message ids and references, and collapse quoted history.	MUST
FR-E4	Classify each thread as reply needed, waiting, informational, or ignore, with a confidence score.	MUST

ID	REQUIREMENT	PRIORITY
FR-E5	Extract asks, people, and dates, and propose tasks and memories for confirmation.	MUST
FR-E6	Draft replies in the owner's voice and store them in the local outbox with a content hash.	MUST
FR-E7	Send an approved draft over SMTP exactly once, with the approved hash and recipient list.	SHOULD
FR-E8	Never mark, move, label, or delete mail in v1.0.	MUST
FR-E9	Show attachment metadata locally and never upload attachments to a channel or cloud model.	MUST

13.3 Calendar

ID	REQUIREMENT	PRIORITY
FR-C1	Fetch and parse one or more private iCal URLs, with per-calendar health status.	MUST
FR-C2	Expand recurrence rules and time zones locally, including exceptions.	MUST
FR-C3	Detect conflicts and propose free slots within configured working hours.	MUST
FR-C4	Generate a valid `.ics` invite draft for review; never create an event directly.	MUST
FR-C5	Include calendar titles and attendees in local surfaces only, never in Telegram messages.	MUST

13.4 Tasks

ID	REQUIREMENT	PRIORITY
FR-T1	Create, edit, complete, cancel, and reopen tasks with the fields in Section 9.1.	MUST
FR-T2	Parse natural-language due dates and times in the local time zone, asking one question when ambiguous.	MUST
FR-T3	Create explicit "remind me" tasks immediately with a one-line confirmation.	MUST
FR-T4	Propose extracted tasks and require confirmation before they enter the ledger.	MUST
FR-T5	Deduplicate by normalised title, source, and due window, offering an update instead of a duplicate.	MUST
FR-T6	Support recurrence with local expansion and no double-firing of completed instances.	SHOULD
FR-T7	Deliver reminders through the active channel, respecting quiet hours except for urgent tasks.	MUST

13.5 Briefs and follow-ups

ID	REQUIREMENT	PRIORITY
FR-B1	Generate a morning brief from structured data, with at most three must-dos and one thing to ignore.	MUST
FR-B2	Render the brief without a model; the model may only add connective prose.	MUST
FR-B3	Detect unanswered sent mail older than a configurable threshold and add it to waiting-on.	MUST
FR-B4	Produce an evening brief and a weekly review with completions, slippage, and open loops.	SHOULD
FR-B5	Suppress duplicate briefs and respect quiet hours and mute commands.	MUST

13.6 Telegram channel

ID	REQUIREMENT	PRIORITY
FR-G1	Connect a bot via long polling and accept messages only from the allowlisted chat id.	MUST
FR-G2	Support `/brief`, `/tasks`, `/task`, `/approve`, `/deny`, `/mute`, and `/status`.	MUST
FR-G3	Render approvals as inline buttons bound to a nonce, expiry, and payload hash, usable once.	MUST
FR-G4	Send summaries only: no email bodies, attachments, attendee lists, or contact details.	MUST
FR-G5	Apply quiet hours, rate limits, and deduplication to every outbound message.	MUST
FR-G6	Record every outbound message with template, approval id, and delivery status.	MUST
FR-G7	Accept voice notes and transcribe locally. Deferred to the voice milestone.	COULD

13.7 Desk and approvals

ID	REQUIREMENT	PRIORITY
FR-D1	Show a run timeline with agents, tools, tokens, cost, duration, and outcome.	MUST
FR-D2	Show a pending approvals queue with payload preview, scope, source, and expiry.	MUST
FR-D3	Approve, reject, revoke, and edit a proposal; an edit returns it to proposed.	MUST
FR-D4	Show the outbox with draft state, recipient count, and send result.	MUST
FR-D5	Show connector health, backlog, and a manual resync.	MUST
FR-D6	Provide a filterable audit log and JSON export.	MUST

ID	REQUIREMENT	PRIORITY
FR-D7	Reject expired, replayed, or hash-mismatched approvals with a visible reason.	MUST

13.8 Memory and Brain

ID	REQUIREMENT	PRIORITY
FR-M1	Capture memories with type, text, sensitivity, confidence, and at least one source.	MUST
FR-M2	Propose captures and require confirmation, except for an explicit "remember that…".	MUST
FR-M3	Retrieve with keyword, recency, type, and graph proximity, filtered by sensitivity first.	MUST
FR-M4	Supersede without deleting; show history on demand.	SHOULD
FR-M5	Forget a memory and remove its source edge, recording the action without content.	MUST
FR-M6	Operate with zero notes and zero connectors, showing an onboarding state instead of an error.	MUST

13.9 Galaxy

ID	REQUIREMENT	PRIORITY
FR-X1	Serve a runtime graph from memories, people, projects, tasks, events, sources, and connectors.	MUST
FR-X2	Support Explore, Today, Open loops, People, Provenance, and Recent modes.	SHOULD
FR-X3	Open a side panel with evidence and a link to the Desk on node click.	MUST
FR-X4	Bound node and edge counts, cache briefly, and remain responsive at 2,000 nodes.	MUST
FR-X5	Keep the optional notes mode working when the vault exists.	COULD

13.10 Files, apps, browser, and scripts

ID	REQUIREMENT	PRIORITY
FR-F1	Operate only inside declared roots and refuse paths outside them, including via symlinks.	MUST
FR-F2	Produce a dry-run plan with counts and sizes before any batch change.	MUST
FR-F3	Move deletions to the Recycle Bin and record every operation.	MUST
FR-F4	Launch allowlisted applications and save an arrangement as a named routine.	SHOULD
FR-F5	Use a dedicated browser profile with a domain allowlist and block payment, banking, and health domains.	MUST

ID	REQUIREMENT	PRIORITY
FR-F6	Require approval before submit, upload, booking, or purchase in a browser session.	MUST
FR-F7	Run only manifested scripts with typed arguments, a timeout, and captured output.	MUST
FR-F8	Never execute arbitrary shell produced by a model.	MUST

13.11 Agents and routing

ID	REQUIREMENT	PRIORITY
FR-A1	Ship the core six agents with the ceilings and lanes in Section 10.2.	MUST
FR-A2	Intersect agent permission with parent authority and turn scope on every call.	MUST
FR-A3	Enforce depth limit 2 and per-turn token, time, and tool budgets.	MUST
FR-A4	Validate agent output against a schema; reject and report malformed proposals.	MUST
FR-A5	Route deterministically where intent is clear; use a model only when ambiguous.	SHOULD

14 Non-functional requirements

The qualities that decide whether the assistant is trusted after the novelty wears off.

14.1 Performance

METRIC	TARGET	MEASUREMENT
Home first render	Under 1.2 s warm, under 3 s cold	Local Chrome, no network
Brief generation	Under 2 s without model; under 8 s with local model	Structured data path
Email incremental sync	Under 10 s for 50 new messages	IMAP fetch plus parse
Local draft, 150 words	Under 12 s on the RTX 3050	Ollama, 3B–4B Q4
Galaxy at 2,000 nodes	60 fps pan, under 2 s load	DevTools performance trace
Approval round trip	Under 2 s from tap to audit row	Telegram callback

14.2 Reliability

- The assistant starts and serves Home, Brain, tasks, and memory with no provider key, no Ollama, and no network.
- Connector failures are isolated: one broken connector must not block the brief or the task ledger.

- Every write is transactional. A failed send leaves the draft in *failed*, not *sent*.
- Crash recovery resumes from the last committed cursor; no duplicate tasks or messages on restart.
- A nightly backup copies the database and configuration to a local folder, excluding secrets.

14.3 Security

- Server binds to `127.0.0.1` only. No CORS wildcard. No served path can reach `config.json`, `data/`, or the vault.
- All external input is schema-validated and size-limited. Path traversal and symlink escape are rejected.
- Approval payloads are immutable and hash-bound. Nonces are single-use and expire.
- Logs redact secrets by key and pattern; a test asserts no credential value appears in any log line.
- Dependency updates are pinned and reviewed; no post-install scripts from untrusted packages.

14.4 Usability

- The daily loop — read brief, work queue, capture, close day — is completable without a manual.
- Every error states what failed, what was preserved, and the next action.
- Keyboard-first on desktop; every primary action reachable without a mouse.
- No modal chains. No more than one decision per screen region.
- Contrast meets WCAG AA; the UI remains legible at 125% scaling.

14.5 Maintainability

- Typed Python with `ruff` and `mypy` in CI; tests for every connector and policy rule.
- Connector and channel adapters are replaceable without touching the orchestrator.
- Schema migrations are versioned, forward-only, and tested against a copy of real data shapes.
- No feature depends on the test notes in `notes/`.

14.6 Accessibility and tone

The assistant speaks plainly, admits uncertainty, and never flatters. It uses short sentences, avoids exclamation marks in system messages, and states limitations before offering workarounds. Screen-reader labels are required on all controls, and the Galaxy has a list-view equivalent.

> **DEFINITION OF DONE**
>
> A requirement is done when it has a test, a visible surface, an audit event where relevant, and a line in the operator documentation. Code alone is not done.

SECTION 15 ROADMAP AND MILESTONES

15 Roadmap and milestones

Six milestones, each independently useful, each ending with something the owner can actually use that week.

15.1 Milestone plan

MILESTONE	PHASES	OUTCOME	DEPENDS ON
M1 Privacy + data	2E.0 connector foundation 2M local model and router 2E.1 email read 2E.2 calendar read	Real mail and calendar data, ingested and processed entirely locally, with a visible privacy lane.	v0.7.1
M2 Control + contact	2D Desk 2J Telegram channel	Runs, approvals, audit, and phone control. The owner can see and steer the system.	M1
M3 Draft + act	2F outbox 2G actions and grants	Drafts, approved sends, and scoped automation with a clean audit trail.	M2
M4 Proactive + tasks	2H briefs and scheduler 2K task ledger and follow-ups	Morning brief, reminders, waiting-on tracking, and a daily plan that reflects reality.	M3
M5 Automation	3D files and apps 3E browser tasks 3F scripts	Downloads cleaned, routines run, forms prepared, research gathered — all with approval.	M4
M6 Product	3G voice 3H packaging and onboarding 3I hardening and v1.0	A daily-usable, installable v1.0 with backup, recovery, and documentation.	M5

T-3 — Milestone plan. Phase numbers align with the implementation plan; the task ledger moves earlier than originally planned because briefs depend on it.

15.2 Phase detail and exit criteria

PHASE	BRANCH	EXIT CRITERIA
2E.0	`phase-2e0-connector-foundation`	Connector interface, registry, health, capability enforcement, fixtures, and tests. No credentials required.
2M	`phase-2m-local-model-router`	Ollama adapter, lane routing, refusal path, and tests proving private content cannot reach cloud.
2E.1	`phase-2e1-email-read`	IMAP sync, thread reconstruction, local classification, proposal queue. Read-only verified by test.
2E.2	`phase-2e2-calendar-read`	iCal ingestion, recurrence expansion, conflict detection, free-slot search.
2D	`phase-2d-desk`	Runs, approvals, outbox, connectors, and audit in one surface.
2J	`phase-2j-telegram`	Allowlisted bot, commands, briefs, approval buttons, quiet hours, outbound audit.

PHASE	BRANCH	EXIT CRITERIA
2F	`phase-2f-outbox`	Draft CRUD, hash binding, edit invalidation, send-once semantics.
2G	`phase-2g-actions-grants`	Approved execution, grant model, revocation, and audit completeness.
2H	`phase-2h-briefs`	Morning, evening, and weekly briefs; follow-up detection; quiet hours.
2K	`phase-2k-tasks`	Ledger, natural-language dates, recurrence, dedupe, reminders, Home integration.
3D	`phase-3d-os-automation`	Allowlisted roots, dry-run plans, Recycle Bin deletes, app routines, manifests.
3E	`phase-3e-browser`	Playwright profile, domain allowlist, research capture, form preparation, submit approval.
3F	`phase-3f-scripts`	Script manifest, typed args, timeouts, output capture, per-run approval.
3G	`phase-3g-voice`	Local STT and TTS, voice notes over Telegram, push-to-talk in the UI.
3H	`phase-3h-packaging`	Installer, first-run onboarding, backup and restore, diagnostics bundle.
3I	`phase-3i-hardening`	Threat-model review, performance pass, accessibility pass, v1.0 tag.

15.3 Working method

- Each phase is a descriptively named branch, reviewed as a PR, squash-merged, then the branch is deleted and the repository returns to `main`.
- Work stops after a PR is opened unless the owner explicitly asks to continue.
- Every phase ends with tests, documentation, and a short note in the implementation plan.
- No phase may depend on a credential to be reviewable; fixtures and mocks are required.

15.4 Critical path

The critical path is **2E.0 → 2M → 2E.1 → 2D → 2J → 2K → 2H**. That sequence produces the daily loop: read mail locally, see approvals, approve from the phone, capture tasks, receive a brief. Everything else improves quality of life around that loop.

SECTION 16 RISKS AND MITIGATIONS

16 Risks and mitigations

The risks that could make the product useless, unsafe, or abandoned — and the decision that reduces each one.

#	RISK	LIKELIHOOD	IMPACT	MITIGATION
R1	Google retires app passwords for this account, breaking IMAP.	Medium	High	Connector interface isolates auth; Gmail API OAuth adapter is the fallback; document the seven-day Testing-mode limitation.
R2	The 3B–4B local model is too weak for useful drafts.	Medium	Medium	Deterministic templates, structured outputs, retrieval of past sent mail as style examples, and an easy path to a 7B model with CPU offload.
R3	Approval friction causes rubber-stamping or abandonment.	High	High	Batch approvals, clear consequence summaries, one-tap from Telegram, and grants only after a workflow proves stable.
R4	Prompt injection through email or web content.	Medium	High	Untrusted envelope, deterministic tools, no auto-fetch, no model-selected recipients, approval on every write.
R5	Private content leaks to the cloud through a fallback path.	Low	Critical	Lane enforcement in the router, not in prompts; tests that assert refusal; no cloud fallback on local failure.
R6	The private iCal URL is rotated or leaked.	Medium	Medium	Credential vault storage, health check with a clear error, one-command rotation, and no URL in logs.
R7	Scope creep turns v1.0 into an unfinished platform.	High	High	Section 3 boundary, milestone exits, and a rule that new agents or connectors need three repeated workflows.
R8	Browser and OS automation is fragile across app updates.	High	Medium	Prefer structured APIs; allowlisted domains and executables; dry runs; fail visibly rather than improvise.
R9	Mailbox size makes local sync and storage expensive.	Medium	Medium	Incremental sync, 90-day retention, attachment-on-demand, and a hard cache size cap with eviction.
R10	Laptop thermals throttle local inference.	Medium	Low	Small model, short contexts, batch jobs on AC power, and a visible latency budget.
R11	The Galaxy becomes decorative and unused.	Medium	Low	Default to Today mode, tie nodes to real open loops, and measure weekly use; remove it if it does not earn its place.
R12	Telegram is unavailable or blocked.	Low	Medium	Desktop notifications remain a first-class channel; the channel interface allows a second adapter.

16.1 Risk review cadence

Risks are reviewed at each milestone exit. A risk that has not changed in two milestones is either accepted in writing or removed. A new critical risk pauses feature work until it has a mitigation.

STOP CONDITIONS

Stop and reassess if any of these occur: private content reaches a cloud provider; an external write executes without an approval record; a connector silently changes read state; or the daily loop takes longer than the manual alternative for two consecutive weeks.

17 Acceptance criteria and metrics

v1.0 is accepted when the owner can live a normal day inside it, and the system proves it kept its promises.

17.1 Scenario acceptance

#	SCENARIO	PASS CONDITION
A1	Cold start with no credentials	Home, Brain, tasks, and memory load; onboarding explains what to connect; no error dialogs.
A2	Connect Gmail and sync	Messages appear locally; read state is unchanged in Gmail; sync completes under the performance target.
A3	Triage a real thread	A reply is proposed with extracted ask, person, and date; the task requires confirmation.
A4	Draft and approve from Telegram	Draft is created locally; approval from the phone sends once; the audit row links draft hash and approval id.
A5	Replay an approval	The second tap is rejected, the user sees why, and the audit records the attempt.
A6	Capture a promise	One line creates the task with person and due date; a single confirmation is shown; no duplicate exists.
A7	Morning brief	Brief renders with no model and with the model; both versions list events, three must-dos, waiting-on, and one ignore.
A8	Find a meeting slot	Three conflict-free slots and an `.ics` draft are produced; no event is created remotely.
A9	Private content with cloud selected	The router refuses the cloud lane and completes locally, or explains the limit. No request is sent.
A10	Injection attempt in email	The directive is surfaced as untrusted, no tool is called, and the user sees a warning.
A11	Clean Downloads	A dry-run plan is shown; approval executes once; deletions land in the Recycle Bin; every operation is audited.
A12	Delete the notes vault	The Galaxy still renders from the life model; nothing errors; the optional notes mode is simply absent.

17.2 Product metrics

METRIC	DEFINITION	TARGET AT 30 DAYS
Daily loop completion	Days with a brief opened and at least one queue item resolved.	≥ 20 of 30
Time to orientation	Time from opening Home to first action.	Under 60 s

METRIC	DEFINITION	TARGET AT 30 DAYS
Proposal precision	Extracted tasks accepted without edit.	≥ 70%
Draft acceptance	Drafts sent with only minor edits.	≥ 50%
Approval latency	Median time from request to decision.	Under 10 min in waking hours
Unapproved writes	External writes without a matching approval.	0
Private egress	Private or sensitive payloads sent to a cloud provider.	0
Brief reliability	Days the brief was delivered on time.	≥ 95%
Open-loop age	Median age of waiting-on items.	Under 5 days

17.3 Quality gates

- All tests pass on Windows 11 with Python 3.14 and no network access for the offline suite.
- No high-severity finding in the dependency audit; no secret in the repository history.
- Every new endpoint has validation tests and a documented error shape.
- Every new tool has a permission test proving it cannot exceed its tier.
- Manual pass of scenarios A1–A12 on a clean profile before the v1.0 tag.

SECTION 18 DECISIONS AND OPEN QUESTIONS

18 Decisions log and open questions

The decisions that shape the build, with the reason they were made — and the questions still worth asking.

18.1 Decisions

#	DECISION	RATIONALE	REVISIT
D1	Gmail personal via IMAP and app password.	Fastest fully local path; no cloud app registration or verification.	If app passwords end
D2	Read-only email first.	Eliminates the largest class of irreversible mistakes while trust is built.	After 2G
D3	Calendar read via private iCal.	No OAuth, no weekly re-auth, enough for briefs and scheduling suggestions.	When writes are needed
D4	Local task ledger.	Owns the model, avoids sync dependencies, exports cleanly later.	If a sync target is requested
D5	Telegram as first channel.	Free, reliable, supports buttons and voice notes, no inbound port.	If blocked

#	DECISION	RATIONALE	REVISIT
D6	Memory-first; notes optional.	The life model must survive deletion of the test vault.	Never
D7	Galaxy as world-model explorer.	A notes graph would become dead weight; a life graph earns daily use.	At M4 review
D8	Windows 11 only.	Matches the machine and keeps OS automation and packaging honest.	Post-v1.0
D9	Local model for private content.	The privacy boundary includes the model, not just storage.	Never without consent
D10	3B–4B model plus deterministic pipelines.	Fits 4 GB VRAM; small models are unreliable at multi-step tool use.	If draft quality fails
D11	Reads and local drafts automatic; external writes approved.	Preserves flow while keeping every irreversible step deliberate.	After 30 days of clean audit
D12	No grants at launch.	Grants are earned by repeated, reliable workflows, not assumed.	At M3 exit
D13	Six agent families, core six agents.	Function-based taxonomy survives app changes; a small roster stays debuggable.	Quarterly
D14	Domain operators, not one god-operator.	Narrow tool scope is the cheapest permission control.	Never
D15	No financial or health automation.	The downside is unbounded and the benefit is small.	Never
D16	Credentials in Windows Credential Manager.	Keeps secrets out of config, database, logs, and git.	Never
D17	No smart home in v1.0.	Different integration surface, low daily value relative to cost.	Post-v1.0

18.2 Open questions

#	QUESTION	NEEDED BY	DEFAULT IF UNANSWERED
Q1	Which exact local model — Qwen 2.5 3B, Qwen 2.5 4B, or Llama 3.2 3B?	Phase 2M	Benchmark all three on ten real triage tasks; pick the best at equal latency.
Q2	What are the working hours and quiet hours?	Phase 2H	09:00–18:00 working; 21:30–07:00 quiet.
Q3	Which folders may the file operator touch?	Phase 3D	Downloads and a staging folder only.
Q4	How long should mail bodies be retained?	Phase 2E.1	90 days, with per-thread deletion.
Q5	Should sent mail be read to learn voice?	Phase 2E.1	Yes, read-only, never quoted outward.

#	QUESTION	NEEDED BY	DEFAULT IF UNANSWERED
Q6	Which cloud provider stays configured for non-sensitive work?	Phase 2M	Keep the current NVIDIA NIM configuration; make it swappable.
Q7	Is a 7B model acceptable if 3B drafts disappoint?	Phase 3I	Yes, for batch triage only, with CPU offload.
Q8	Should the Galaxy remain 3D or offer a list-first view?	Phase 2C.2	Keep 3D, add a list view for accessibility and speed.

APPROVAL

This draft is ready for review. Once approved, the next action is to open `phase-2e0-connector-foundation` and implement Section 7.1 with tests. No credentials are needed for that phase.

APPENDIX A GLOSSARY

A Glossary

Shared vocabulary for the build. If a term is not here, it probably should not be in a requirement.

TERM	DEFINITION
Approval	A bound, expiring, single-use decision that authorises one exact payload to be executed by one tool.
Audit event	An append-only record of something the system read, proposed, decided, or did. Metadata only.
Channel	A transport for messages to or from the owner: Telegram, desktop, later SMS or voice.
Connector	An adapter to one external system with declared read, draft, write, and delete capabilities.
Desk	The control surface: runs, approvals, outbox, connector health, audit.
Draft	A locally stored, unsent message or invite with a content hash.
Grant	Scoped, expiring permission to perform a class of actions without per-action approval.
Life model	The structured set of memories, people, projects, tasks, events, and sources that the assistant reasons over.
Lane	The permitted reasoning location for a request: local, cloud, or refusal.
Memory	A durable fact, preference, decision, or observation with provenance and sensitivity.
Open loop	Anything waiting on the owner or on someone else: an unanswered mail, a promise, a blocked task.
Orchestrator	The single component that owns a turn, dispatches agents, and requests approval.
Proposal	A structured suggestion — task, memory, draft, or action — that is not yet committed.

TERM	DEFINITION
Run	One orchestrated turn, with agents, tools, tokens, cost, duration, and outcome.
Sensitivity	The privacy class of a record: public, internal, private, sensitive, or secret.
Source	The email, message, note, document, or meeting that a record derives from.
Subagent	A scoped worker with a permission ceiling, a lane, a schema, and budgets.
Untrusted envelope	A wrapper that marks external content as data, never instructions, before model input.
World graph	The runtime projection of the life model served to the Galaxy.

A.1 Naming

The product, repository, package, and persona are all **A.S.H.R.A**. Internal identifiers use `ashra` in lower case. The earlier working name "myjarvis" appears only in the folder path and is not a product name.

APPENDIX B CONFIGURATION REFERENCE

B Configuration reference

Non-secret configuration lives in `config.json`. Secrets live in the vault and are referenced by name.

MODEL AND PRIVACY

```
{
  "provider": "ollama",
  "model": "qwen2.5:3b",
  "api_base": "http://127.0.0.1:11434/v1",
  "cloud": {
    "provider": "nvidia",
    "model": "nvidia/nemotron-3.5-super-vl-120b-a12b",
    "credential_ref": "ashra/cloud"
  },
  "privacy": {
    "default_lane": "local",
    "allow_cloud_for": ["public", "internal"],
    "refuse": ["secret"]
  }
}
```

CONNECTORS

```
{
  "connectors": {
    "gmail": {
      "enabled": true,
      "credential_ref": "ashra/gmail",
      "mode": "read",
      "folders": ["INBOX"],
      "retention_days": 90
    },
    "gcal": {
      "enabled": true,
      "credential_ref": "ashra/calendar",
      "mode": "read",
      "window_back_days": 60,
      "window_forward_days": 180
    }
  }
}
```

CHANNELS, TASKS, AND AUTOMATION

```
{
  "channels": {
    "telegram": {
      "enabled": true,
      "credential_ref": "ashra/telegram",
      "allowlist_chat_ids": ["<your-chat-id>"],
      "quiet_hours": "21:30-07:00"
    }
  },
  "tasks": {
    "default_reminder": "09:00",
    "max_high_priority_per_day": 3
  },
  "automation": {
    "file_roots": ["%USERPROFILE%/Downloads", "%USERPROFILE%/AshraStaging"],
    "delete_mode": "recycle",
    "grants": []
  }
}
```

B.1 Rules

- `config.json` is gitignored and never served. The server rejects any request path that resolves to it.
- A `credential_ref` is a name, not a value. The vault resolves it at call time.
- The local lane is the default. Cloud is opt-in per sensitivity class, never per prompt.
- Invalid or unknown keys fail startup with a precise message; the system does not guess.
- Configuration is re-read per request where safe, so a model change does not require a restart.

B.2 Environment

ITEM	VALUE
OS	Windows 11 Home, 10.0.26300
CPU / RAM	Intel i5-12500H, 16 GB
GPU	NVIDIA RTX 3050 Laptop, 4 GB VRAM
Python	3.14.7
Local runtime	Ollama on `127.0.0.1:11434`
Server	Uvicorn on `127.0.0.1:4700`
Database	SQLite at `data/ashra.db`

C Phase history

What already exists at the time of this draft. The baseline is v0.7.1 at commit `5b774ac`, with 171 tests passing.

PHASE	SCOPE	STATE
0	Foundation: rename to A.S.H.R.A, package layout, config validation, tests, CI, small talk without a key.	Complete
1	FastAPI and SQLite, sessions, audit, permission tiers, filesystem tools, SSE streaming, MCP server.	Complete
1B	Assistant pipeline: note index, retrieval, provider transport, chat surface, capture.	Complete
1C	Orchestrator: bounded tool loop, approvals, proposal propagation.	Complete
1D	MCP integrations: client, configuration, manager, tool registration.	Complete
1E	Scoped subagents: registry, permission ceilings, child runner.	Complete
2A	Typed memory substrate: lifecycle, retrieval, graph, sensitivity.	Complete
2B	Brain UI: capture, review, search, provenance.	Complete
2C	Home hub: Needs You queue, brief region, capture loop.	Complete
2C.1	Galaxy load performance: local vendor assets, caching.	Complete
2E.0	Connector foundation.	Next
2M	Local model and privacy router.	Planned

C.1 What this means for the reader

The foundation is not a prototype. Sessions, approvals, audit, memory, agents, and the Home hub already exist and are tested. The work ahead is mostly connective: real data, a privacy router, a control channel, and the task ledger that turns observation into action.

C.2 Immediate next steps

1. **Approve the scope.** Confirm Sections 3, 6, and 9. Anything not listed as in scope waits.
2. **Open the connector foundation.** Branch `phase-2e0-connector-foundation`; implement the contract in Section 7.1 with fixtures and tests. No credentials required.
3. **Install the local runtime.** Ollama, then benchmark Qwen 2.5 3B, Qwen 2.5 4B, and Llama 3.2 3B on ten real triage tasks; choose on quality at equal latency (Q1).
4. **Prepare credentials locally.** Gmail app password, calendar iCal address, and Telegram bot token go into Windows Credential Manager through a setup command — never into chat, config, or git.
5. **Set the daily defaults.** Working hours, quiet hours, reminder time, and the two file roots the operator may touch (Q2, Q3).

> **DOCUMENT END**
>
> A.S.H.R.A Product Requirements Document, version 1.0 — Draft, 13 November 2026. Prepared for local review. Not for distribution or publication.